

```

/* L2CAP socket address */
struct sockaddr_l2 {
    sa_family_t    l2_family;
    unsigned short l2_psm;
    bdaddr_t       l2_bdaddr;
};

```

Figure 10: Structure *sockaddr_l2*

```

/* RCOMM socket address */
struct sockaddr_rc {
    sa_family_t    rc_family;
    bdaddr_t       rc_bdaddr;
    unit8_t        rc_channel;
};

```

Figure 11: Structure *sockaddr_rc*

RFCOMM and L2CAP servers

RFCOMM and L2CAP have socket APIs to open socket descriptors like TCP/IP programming. But here we have the Bluetooth family's *AF_BLUETOOTH*.

 **Note:** For RFCOMM sockets, include the *bluetooth/rfcomm.h* header file and *bluetooth/l2cap.h* for L2CAP, as well as *bluetooth/bluetooth.h* and *sys/socket.h* in both.

RFCOMM has the *SOCK_STREAM* type and *BTPROTO_RFCOMM* protocol:

```
s = socket (AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM);
```

L2CAP has the *SOCK_SEQPACKET* type and *BTPROTO_L2CAP* protocol:

```
s = socket ( AF_BLUETOOTH , SOCK_SEQPACKET , BTPROTO_L2CAP );
```

TCP/IP has the *sockaddr_in* structure and similarly, L2CAP has the *sockaddr_l2* structure while RFCOMM has the *sockaddr_rc* structure (see Figures 10 and 11).

 **Note:** After kernel 2.4.37, the *sockaddr_l2* structure has some extra fields like *l2_cid* and *l2_bdaddr_type*.

Use the *BADDR_ANY* macro so that the server can listen to all local Bluetooth adapters. Bluetooth uses *htobs* and *btohs* instead of *htons* and *ntohs*, which are used in TCP/IP.

For RFCOMM, type:

```

loc_addr . rc_family = AF_BLUETOOTH;
loc_addr . rc_bdaddr = BADDR_ANY;
loc_addr . rc_channel = 1;

```

For L2CAP, type:

```
loc_addr . l2_family = AF_BLUETOOTH;
```

```

loc_addr . l2_bdaddr = BADDR_ANY;
loc_addr . l2_psm = htobs(0x1001);

```

Since you know the TCP/IP concepts, you can easily understand the next APIs mentioned like *bind*, *listen*, *accept*, *send* and *recv*.

For RFCOMM and L2CAP, type:

```

bind ( s , ( struct sockaddr * ) &loc_addr , sizeof ( loc_addr ) );
listen ( s , 1 ); //for one connection
client = accept ( s , ( struct sockaddr * ) &rem_addr ,
&(sizeof( rem_addr )) );

```



Note: Before printing the received Bluetooth address, convert it to a string by using the *ba2str* API, which is described earlier in this article. You can also print the address by accessing the *bdaddr_t* structure.

Now receive data from the client:

```

b = recv ( client , buf , sizeof ( buf ) , 0 );
if ( b > 0 ) {
    printf ( "received [%s]\n " , buf );
}

```

And after reading the data, close the sockets.

```

close ( client );
close ( s );

```

RFCOMM and L2CAP client

For the client, first open the socket.

For RFCOMM, type:

```
s = socket (AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM);
```

For L2CAP, type:

```
s = socket ( AF_BLUETOOTH , SOCK_SEQPACKET , BTPROTO_L2CAP );
```

Now connect this socket with the existing server.

For RFCOMM, type:

```

struct sockaddr_rc addr;
char dest[18] = "<Bluetooth Address>" //put server's
bluetooth adapter address
addr . rc_family = AF_BLUETOOTH;
addr . rc_psm = 1;
str2ba ( dest , &addr . rc_bdaddr );
check = connect ( s , ( struct sockaddr * ) &addr , sizeof ( addr ) );

```

For L2CAP, type:

To be continued on page no. 58...